



STUDENT GUIDE
2026 EDITION

SINICA × NCHC

CUDA-Q Agentic Coding Bootcamp

學生實作指南

DATE & TIME

2026 年 8 月 7 日
09:30–16:30

CLASSROOM

NCHC H200 VM
CUDA-Q + Agentic AI

ORGANIZED IN COLLABORATION WITH



中央研究院
ACADEMIA SINICA



中央研究院 關鍵議題研究中心
Research Center for Critical Issues | Academia Sinica



NIAAR NATIONAL INSTITUTES OF APPLIED RESEARCH
National Center for
High-performance Computing



OPEN
HACKATHONS
powered by OpenACC



NVIDIA

活動資訊

由中央研究院關鍵議題研究中心（RCCI）、國家高速網路與計算中心（NCHC）、NVIDIA 與 OpenACC Organization 合作舉辦的一日實作課程。學員將使用 NCHC H200 環境，從 CUDA-Q 基礎、NVIDIA Ising 到 NCHC RAP 與 Agentic Coding 完成量子運算實作。

DATE & TIME

2026 年 8 月 7 日（星期五）
09:30-16:30（臺灣時間）

FORMAT

實體課程
In-Person Bootcamp

VENUE

中央研究院南部院區
關鍵議題研究中心大樓 1 樓演講廳
臺南市歸仁區歸仁十三路一段 100 號



中央研究院
ACADEMIA SINICA



中央研究院 關鍵議題研究中心
Research Center for Critical Issues | Academia Sinica



NCHC NATIONAL INSTITUTES OF APPLIED RESEARCH
National Center for
High-performance Computing



OPEN
HACKATHONS
powered by OpenACC



NVIDIA

COURSE AI SERVICE

NCHC RAP • Agentic Coding 模型服務



TAIWAN AI RAP

OFFICIAL LINKS

[活動報名頁](#) • [Open Hackathons](#)

[活動資訊與議程](#) • [nqobu/nvidia/20260807](#)

[CUDA-Q Agentic Coding 教材](#) • [GitHub main](#)

[課程 AI 服務](#) 活動 [Taiwan AI RAP](#) poster token、參考答案或講師映像封裝資訊。文件中的 IP 僅為示範。

Agenda 議程

*All times are in Taiwan Standard Time (台灣標準時間)

Friday, August 7, 2026 // 09:30 AM - 04:30 PM (In-Person) (實體活動)

TIME 時間	SESSION 課程內容
● 09:30 AM – 10:00 AM 上午 09:30 – 10:00	Registration & Setup 報到與環境設定
● 10:00 AM – 10:15 AM 上午 10:00 – 10:15	Welcome & Introduction (NCHC + NVIDIA) 歡迎致詞與課程介紹 (NCHC + NVIDIA)
● 10:15 AM – 10:45 AM 上午 10:15 – 10:45	NVIDIA Quantum Platform Overview: CUDA-Q & NVIDIA Ising NVIDIA 量子運算平台總覽：CUDA-Q 與 NVIDIA Ising
● 10:45 AM – 11:15 AM 上午 10:45 – 11:15	CUDA-Q Environment Setup CUDA-Q 開發環境設定
● 11:15 AM – 12:00 PM 上午 11:15 – 下午 12:00	Basic CUDA-Q Hands-on & NVIDIA Ising Calibration Playground 基礎 CUDA-Q 實作與 NVIDIA Ising 校準體驗
● 12:00 PM – 01:00 PM 下午 12:00 – 下午 01:00	 Lunch Break 午餐時間
● 01:00 PM – 02:00 PM 下午 01:00 – 下午 02:00	Superconducting QPU Introduction & Agentic Calibration 超導量子處理器介紹與 Agentic 校準
● 02:00 PM – 02:30 PM 下午 02:00 – 下午 02:30	NCHC RAP Service Briefing & Environment Setup NCHC RAP 服務介紹與環境設定
● 02:30 PM – 03:15 PM 下午 02:30 – 下午 03:15	Advanced CUDA-Q Hands-on with RAP 進階 CUDA-Q 實作 (結合 RAP 服務)
● 03:15 PM – 04:00 PM 下午 03:15 – 下午 04:00	Bring-Your-Own-Code with CUDA-Q Agentic Coding 自備程式碼實作：CUDA-Q Agentic Coding
● 04:00 PM – 04:20 PM 下午 04:00 – 下午 04:20	Q&A & Open Discussion 問答與綜合討論

目錄

完整連結可直接點閱 • 各 Step 皆從新頁開始

- 你會使用什麼

- Step 1：使用指定 IP 與 `bootcamp0807.pem` 登入 VM

macOS

Windows 10／11 PowerShell

第一次連線

- Step 2：一鍵啟用、驗證並啟動課程環境

- Step 3：確認 JupyterLab 狀態

- Step 4：從筆電直接開啟 JupyterLab

備案：SSH tunnel (SSH port 3322)

- Step 5：認識課程檔案

- Step 6：先完成 CUDA-Q 暖身

- Step 7：在 Terminal 使用 OpenCode

選配：在 Terminal 啟用自動批准

查看今天的 Token 使用量

- Step 8：在 Jupyter AI 使用 `@openCode`

- Step 9：核心 QAOA Agentic Coding 練習

- Step 10：BYOC：Qiskit → CUDA-Q

- 常用維運指令

- 故障排除

- 下課前

你會使用什麼

國網已替你準備好：

- Ubuntu GPU VM
- NVIDIA H200 與 Driver
- Docker 與 NVIDIA Container Toolkit
- CUDA-Q 0.15.0 課程 container
- JupyterLab、Jupyter AI、OpenCode
- `cudaq-agentic-coding` 教材（學生環境不含參考答案）

你不需要安裝 CUDA、不需要 build Docker image，也不需要登入 GitHub。

整體資料流如下：

你的瀏覽器

| HTTP 直接連線 (SSH tunnel 為備案)



國網學生 VM (0.0.0.0:8888, 需 Jupyter token)

| Docker + NVIDIA runtime



CUDA-Q 課程 container

└─ JupyterLab / Jupyter AI

└─ OpenCode + NCHC RAP

└─ /workspace/cudaq-agentic-coding



└─ 掛載自 /home/ubuntu/cudaq-agentic-coding

Step 1：使用指定 IP 與 `bootcamp0807.pem` 登入 VM

國網會預先準備約 40 台 VM。每位學生會取得：

- `bootcamp0807.pem` SSH private key
- 一個指定的 VM IP
- 固定登入帳號 `ubuntu`

以下使用 `140.110.109.XXX` 作為示範；請換成教師分配給你的 IP，不要登入其他同學的 VM。

macOS

1. 將 `bootcamp0807.pem` 放在 `Downloads`。
2. 開啟 Terminal，限制 private key 權限：

```
chmod 600 ~/Downloads/bootcamp0807.pem
```

1. 登入指定 VM：

```
ssh -p 3322 -i ~/Downloads/bootcamp0807.pem ubuntu@140.110.109.XXX
```

Windows 10／11 PowerShell

Windows 10／11 通常已內建 OpenSSH Client。開啟 PowerShell：

```
ssh -p 3322 -i "$HOME\Downloads\bootcamp0807.pem" ubuntu@140.110.109.XXX
```

如果顯示 `ssh is not recognized`，請至：

```
Settings → Apps → Optional Features → Add a feature → OpenSSH Client
```

安裝完成後重新開啟 PowerShell。

第一次連線

第一次連線可能看到：

```
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

確認 IP 是教師分配的 VM 後輸入：

```
yes
```

成功後，提示字元會類似：

```
ubuntu@vm206-2:~$
```

Step 2：一鍵啟用、驗證並啟動課程環境

```
cd ~/cudaq-course-kit  
./activate-student-course.sh
```

腳本會依序：

1. 從 VM image 的活動憑證包建立你的 `course.env`。
2. 為這台 VM 產生獨立 Jupyter token。
3. 顯示活動 key 課後到期與日後使用個人 key 的通知。
4. 執行 H200、Docker、CUDA-Q 與 Notebook 驗證。
5. 啟動發布在 VM `0.0.0.0:8888`、以獨立 token 保護的 JupyterLab。

最後應看到環境檢查全部 PASS，以及醒目的登入連結：

★ 下一步 / NEXT STEP ★

請在你的筆電瀏覽器開啟以下完整 JupyterLab 登入連結：

OPEN THIS COMPLETE LOGIN LINK FROM YOUR LAPTOP:

`http://你的VM-IP:8888/lab?token=這台VM的獨立token`

若失敗，不要自行安裝或升級 CUDA、Driver、Docker，也不要將 key 貼到聊天；將完整錯誤交給助教。

Step 3：確認 JupyterLab 狀態

```
docker compose \
  --project-directory ~/cudaq-course-kit \
  --env-file ~/.config/nchc-cudaq-course/course.env \
  -f ~/cudaq-course-kit/compose.yaml \
  ps
```

服務狀態應變成 `healthy`。若尚為 `starting`，等待約 10-30 秒再查一次。 `PORTS` 顯示 `0.0.0.0:8888->8888/tcp` 是正確結果：`0.0.0.0` 是 Docker 在 VM 本機的監聽位址，NCHC 的 floating/public IP（例如 `140.110.109.XXX`）屬於外部 NAT 映射，不會出現在 `docker compose ps`。

Step 4：從筆電直接開啟 JupyterLab

`start-lab.sh` 會在終端顯示這台 VM 的完整登入連結，格式如下：若未設定 `JUPYTER_PUBLIC_HOST`，腳本會透過兩個獨立 HTTPS 來源交叉確認這台 VM 的 public IP；只有兩者回傳相同且有效的 IPv4 才採用，因此 40 台 VM 不需逐台填寫。若查詢失敗，才會退回本機 IP 並顯示警告。

```
http://140.110.109.XXX:8888/lab?token=這台VM的獨立token
```

直接在自己的筆電點擊或複製該連結即可登入，不必再手動輸入 token。若自動偵測的 IP 與教師分配的 VM IP 不同，請將網址中的 IP 換成教師提供的 IP，保留其餘部分。TCP 8888 必須由 NCHC 網路規則開放給課程使用者；完整網址含有 token，不要分享、截圖或貼到聊天。

備案：SSH tunnel (SSH port 3322)

只有在筆電無法直接連線 VM 的 8888 時，才使用 tunnel。

macOS Terminal：

```
ssh -p 3322 -i ~/Downloads/bootcamp0807.pem -N \  
-L 8888:127.0.0.1:8888 ubuntu@140.110.109.XXX
```

Windows PowerShell：

```
ssh -p 3322 -i "$HOME\Downloads\bootcamp0807.pem" -N -L 8888:127.0.0.1:8888 ubuntu@140.110.109.XXX
```

Tunnel 執行期間沒有輸出是正常的；保持視窗開啟，再使用 `start-lab.sh` 顯示的 localhost 完整登入連結。

若畫面仍要求 token，可使用 `start-lab.sh` 顯示的 token，或在 VM 查詢；不要貼到聊天或 Notebook：

```
grep '^JUPYTER_TOKEN=' ~/.config/nchc-cudaq-course/course.env
```

Step 5：認識課程檔案

在 JupyterLab 左側檔案瀏覽器會看到：

檔案	用途
<code>_intro_cudaq.ipynb</code>	CUDA-Q 基礎暖身
<code>_intro_Ising_Calibration.ipynb</code>	NVIDIA Ising Calibration 特別單元；Lab 會將 <code>NVIDIA_NIM_API_KEY</code> 同值提供為 <code>NVIDIA_API_KEY</code>
<code>00_notebook.ipynb</code>	已完成的 16-qubit QAOA baseline
<code>cudaq-doc.md</code>	提供 agent 閱讀的 CUDA-Q 參考
<code>AGENTS.md</code>	Agent 建立 Notebook 時必須遵守的規則
<code>helpers.py</code>	共用資料、QUBO、驗證與繪圖函式；不要重寫
<code>my_code.py</code>	Qiskit → CUDA-Q 遷移練習

Step 6：先完成 CUDA-Q 暖身

1. 開啟 `_intro_cudaq.ipynb` 。
2. 從上到下逐格執行。
3. 再開啟 `00_notebook.ipynb`，觀察 CPU 與 GPU target 的差異。
4. 在 JupyterLab Terminal 確認 GPU：

```
nvidia-smi  
python /opt/nchc-cudaq-course/smoke-test-cudaq.py
```

Smoke test 會依序測試 `qpp-cpu` 與 `nvidia`，成功時印出兩行 `PASS`。

Step 7：在 Terminal 使用 OpenCode

在 JupyterLab 選擇 `File → New → Terminal`：

```
cd /workspace/cudaq-agentic-coding
opencode
```

在 OpenCode 中：

```
/models
```

可選模型如下；`/models` 中應使用完整的 provider/model ID：

Backend	Model / <code>/models</code> ID	憑證
NCHC RAP Nemotron Super (預設)	<code>rap-nemotron/NVIDIA-Nemotron-3-Super-120B-A12B</code>	<code>RAP_NEMOTRON_3_ULTRA_API_KEY</code>
NCHC RAP Nemotron Ultra	<code>rap-nemotron/NVIDIA-Nemotron-3-Ultra-550B-A55B</code>	與 Super 共用 <code>RAP_NEMOTRON_3_ULTRA_API_KEY</code>
NCHC RAP Gemma 26B	<code>rap-gemma-26b/gemma-4-26B-A4B-it</code>	<code>RAP_GEMMA_26B_API_KEY</code>
NCHC RAP Gemma 31B	<code>rap-gemma-31b/gemma-4-31B-it</code>	<code>RAP_GEMMA_31B_API_KEY</code>
NVIDIA hosted NIM	<code>nvidia-nim/nvidia/nemotron-3-ultra-550b-a55b</code>	<code>NVIDIA_NIM_API_KEY</code>
OpenCode Zen Nemotron	<code>opencode/nemotron-3-ultra-free</code> (Nemotron 3 Ultra Free)	Free pricing；仍需個人 OpenCode Zen API key

課程預設使用 NCHC RAP Nemotron 3 Super。Super 與 Ultra 位於同一個 `rap-nemotron` provider 並共用同一把 RAP key；可用 `/models` 手動切換 Ultra。若 RAP 當天明顯變慢或暫時不可用，先依 [OpenCode Zen 官方說明](#) 建立個人帳號／API key，再在 OpenCode TUI 執行：

```
/connect
```

選擇 `OpenCode Zen`，貼入你從 OpenCode Zen 帳號取得的個人 API key；接著執行 `/models`，選擇 `opencode/nemotron-3-ultra-free`。不要把 Zen key 貼進聊天、Notebook、shell history 或 `course.env`。連線資料只保存在這台 VM 的 OpenCode runtime，執行 `reset-manual-validation.sh` 時會一起移入備份。

Zen 的 Nemotron 3 Ultra 是限時免費 trial，不保證課程當天仍免費、無限量或沒有 rate limit。這個 endpoint 會記錄試用資料以進行安全管理與產品改善；不要送出個資、機密資料、API key 或 Jupyter token。Jupyter AI @OpenCode 要使用 Zen 時，先在 JupyterLab Terminal 完成一次 `/connect`，再重新開啟 Chat。

選好模型後，先讓 agent 讀教材，不產生檔案：

```
Read @cudaq-doc.md and @AGENTS.md, then give me a short summary of what
each one covers. Just the wrap-up. No code or files yet.
```

再做 GHZ 練習：

```
Write a simple script ghz.py using CUDA-Q to prepare a 20-qubit GHZ state on the GPU
("nvidia" target), sample 1000 shots, and print the counts. Refer to @cudaq-doc.md if needed,
then run it and show me the output.
```

Agent 要執行 shell 或修改檔案時會詢問許可。先讀懂指令與目標，再批准。

選配：在 Terminal 啟用自動批准

OpenCode CLI 可以只為目前這次啟動自動批准未被明確拒絕的操作：

```
cd /workspace/cudaq-agnostic-coding
opencode --auto
```

也可以用非互動方式執行單一 prompt：

```
opencode run --auto "在這裡貼上單一步驟的 prompt"
```

`--auto` 會跳過許多工具確認，可能讓 Agent 自動執行 shell、修改或覆寫檔案、執行 Notebook，甚至透過 shell 存取網路或讀取環境變數。使用前請先備份作業，不要讓 prompt 要求顯示 API key、Jupyter token 或 `course.env`；看到不合理的操作應立即中止。關閉這次 OpenCode CLI 後，下次不加 `--auto` 就會恢復逐項詢問。

Jupyter AI 的 OpenCode Chat 使用 ACP；目前安裝版本沒有 per-chat `--auto` 旗標，因此 Chat 仍依課程的 generated permission config 詢問批准。不要自行修改 `.runtime/opencode.json`，它會在 Lab 重啟時重新產生，而且全域 `allow` 會影響所有 Chat。`--auto` 只改變工具批准流程，不會解除 `AGENTS.md` 的 one-step-at-a-time 規則；仍應一次只貼一個課程步驟。

查看今天的 Token 使用量

完成一段練習後，可在 JupyterLab Terminal 查看最近 24 小時的 OpenCode session、token 與工具使用統計：

```
cd /workspace/cudaq-agentic-coding
opencode stats --days 1
```

若想同時看各模型的分布，可顯示使用次數最多的前 5 個模型：

```
opencode stats --days 1 --models 5
```

常見欄位的意義：

區塊／欄位	代表意義
Sessions	最近 1 天內有活動的 OpenCode 對話數；CLI 與 Jupyter AI @OpenCode 都可能包含在內
Messages	這些 session 中的訊息總數，包含使用者、assistant 與工具往返，不等於 prompt 次數
Days	本次統計涵蓋的天數；--days 1 表示向前回溯最近 24 小時
Input	送進模型的 token，包括 prompt、對話歷史、系統規則、程式碼與工具結果；agent 反覆讀檔或執行工具時會快速增加
Output	模型產生的回答 token；通常遠少於 Input
Cache Read	從 provider 的 prompt cache 重用的 token；若 provider 未回報快取資料會是 0
Cache Write	寫入 provider prompt cache 的 token；若 provider 不支援或未回報會是 0
Avg Tokens/ Session	每個 session 的平均總 token；少數超長對話可能把平均值拉高
Median Tokens/ Session	session 總 token 的中位數，較接近「典型」對話用量
Total Cost / Avg Cost/Day	OpenCode 依 provider 回傳或模型價格資料估算的費用；NCHC RAP 顯示 \$0.00 不代表沒有使用 token，也不是活動額度或帳單的正式依據
TOOL USAGE	read、bash、write、edit 等工具被呼叫的次數與占比；invalid 表示模型曾產生無效或無法解析的工具呼叫，可作為 prompt 或模型穩定性的觀察指標

K 代表千、M 代表百萬，例如 29.0K 約為 29,000 tokens、2.6M 約為 2,600,000 tokens。Token 不是字數：中英文、程式碼與標點的切分方式會因模型 tokenizer 而異，因此不同模型之間只能做趨勢比較。

這份統計來自這台 VM 的 OpenCode 本機 session 資料，不是 NCHC RAP 的官方 帳務／配額頁面。執行 `reset-manual-validation.sh` 清理 OpenCode runtime 後，歷史統計與安裝的課程 skills 也會被移入備份。不要為了查看用量而輸出 API key、`course.env` 或 Jupyter token。

Step 8 : 在 Jupyter AI 使用 @OpenCode

1. 在 JupyterLab Launcher 開啟 Chat 。
2. 輸入 @，確認清單中有 OpenCode 。
3. 先做唯讀練習：

```
@OpenCode Read 00_notebook.ipynb and explain its six code cells in beginner-friendly language.  
Do not modify or execute anything yet.
```

1. 再要求計畫：

```
@OpenCode Read AGENTS.md and the Step 1 prompt in README.md. Propose a plan for  
01_notebook.ipynb. Do not edit files until I approve the plan.
```

1. 檢查計畫符合規則後，才允許 agent 建立、執行與驗證 Notebook 。

Step 9：核心 QAOA Agentic Coding 練習

依 repo README 的 Step 1–4，學生在根目錄建立：

```
01_notebook.ipynb
02_notebook.ipynb
03_notebook.ipynb
04_notebook.ipynb
```

每一步都遵循：

1. 讓 agent 讀 `cudaq-doc.md`、`AGENTS.md`、前一步 Notebook 與 `helpers.py`。
2. 先看計畫。
3. 批准建立 Notebook。
4. 執行前先檢查 code cell。
5. 使用 repo 指定的 600 秒外部 timeout 執行一次前景 `nbconvert`。
6. 確認零 error、唯一 validity assert 通過、最終地圖可視化存在。
7. 不因結果不理想而修改抽樣答案或繞過驗證。

學生 VM 不部署參考答案；只由教師在隔離環境、完成當前步驟後唯讀比較。

Step 10 : BYOC : Qiskit → CUDA-Q

課程 image 已補裝 Qiskit，因此原始程式可直接執行：

```
python my_code.py
```

在 OpenCode 提示：

```
Read my_code.py and migrate it to CUDA-Q as grover_cudaq.py. Use @cudaq-doc.md,  
run on the "nvidia" target, then run both files and explain whether the outputs match.
```

常用維運指令

```
cd ~/cudaq-course-kit

./start-lab.sh      # 啟動
./lab-logs.sh      # 查看即時 log ; Ctrl+C 只離開 log，不會停止服務
./stop-lab.sh      # 停止 JupyterLab container
./verify-environment.sh
```

備份目前工作：

```
cp -a ~/cudaq-agentic-coding \
~/cudaq-agentic-coding-backup-$(date +%Y%m%d-%H%M%S)
```

還原乾淨教材：

```
cd ~/cudaq-course-kit
./reset-manual-validation.sh
```

`reset-manual-validation.sh` 會停止 Lab，將 Jupyter/OpenCode runtime、stats/session/cache/log、OpenCode 安裝的 `cudaq-gpu-opt-skill` / `cudaq-guide` 與驗收 log 移至 `~/manual-validation-backups/`，再呼叫 `course-reset.sh` 還原乾淨教材。根目錄 `SKILL.md`、01-04 Notebook、BYOC / playground 程式、chat、checkpoint、`__pycache__` 與 project-local agent skills 都隨目前 repo 移到 `~/course-backups/`。腳本最後會確認新 workspace 與唯讀 source 完全一致；API key、`course.env` 與 Jupyter token 會保留。

故障排除

現象	檢查	處理
<code>permission denied /var/run/docker.sock</code>	<code>groups</code>	重新登入 VM；仍失敗交給助教處理
Container 看不到 GPU	<code>nvidia-smi</code> 、 <code>docker info</code>	不要重裝 Driver；提供 <code>verify-environment.sh</code> 完整輸出
找不到 course image	<code>docker image ls</code>	母 VM 未正確封裝；學生不要自行 pull/build
8888 已被占用	<code>`ss -ltnp`</code>	<code>grep 8888`</code>
SSH 顯示 <code>Permission denied (publickey)</code>	確認使用指定 IP、SSH port 3322、帳號 <code>ubuntu</code> 與正確 PEM 路徑	macOS 再執行 <code>chmod 600</code> ；Windows 確認檔案未被改名為 <code>.pem.txt</code>
SSH 顯示 Connection timed out	確認 IP、網路與 VPN 要求	不要改 VM；將指定 IP 與錯誤畫面交給助教
SSH 顯示 host key changed	不要直接忽略警告	請助教確認該 IP 是否重建或重新分配後，再依指示更新 <code>known_hosts</code>
Jupyter token 錯誤	查個人 course env	關閉舊分頁，使用正確 token 重新登入
@OpenCode 未出現	<code>opencode --version</code> 、重新啟動 lab	查看 <code>lab-logs.sh</code> ，確認 Jupyter AI 3.0.1 正常載入
RAP 401/403	<code>/models</code> 、確認活動時間	不要把 key 貼入聊天；請教師檢查部署後注入與 key 狀態
Python package 衝突	<code>python -m pip check</code>	不要在 Notebook 自行 <code>pip install -U</code> ；還原課程或找助教
Notebook 超過 550 秒	查看是哪個 cell	停止並回報量測，不強迫產生有效結果

下課前

```
cd ~/cudaq-course-kit  
./stop-lab.sh
```

下載或備份你建立的 Notebook。活動 API key 由教師統一撤銷，不要帶離課程環境。

`bootcamp0807.pem` 是活動用 private key，不得上傳 GitHub、雲端公開連結或貼到聊天。活動結束後依教師指示刪除本機副本。



Ready to build with CUDA-Q

先遵守每次只執行一個 prompt 的課程節奏；完成暖身後，再進入 QAOA、NCHC RAP 與 Bring-Your-Own-Code 實作。

github.com/Squirtle007/cudaq-agentic-coding

openhackathons.org • SINICA × NCHC Bootcamp